UNIT–II

The Relational Model & Relational Algebra and Calculus

1. The Relational Model 01
   - 1.1. Introduction to the Relational Model 01
   - 1.2. Integrity Constraints 07
   - 1.3. Enforcing Integrity Constraints 13
   - 1.4. Querying Relational Data 15
   - 1.5. Logical Database Design 17
   - 1.6. Introduction to Views 20
   - 1.7. Destroying/Altering Tables 26
   - 1.8. Views 31
2. Relational Algebra and Calculus 37
   - 2.1 Relational Algebra 37
     - 2.1.1 Selection, 38
     - 2.1.2 Projection, 38
     - 2.1.3 Union, 38
     - 2.1.4 Difference, 39
     - 2.1.5 Rename, 39
     - 2.1.6 Cartesian Product 40
   - 2.2 Derived Operators 41
     - 2.2.1 Joins 41
       - 2.2.1.1 Inner Join 41
       - 2.2.1.2 Outer Join 41
     - 2.2.2 Intersection 42
     - 2.2.3 Division 42
   - 2.3 Relational Calculus 44
     - 2.3.1 Tuple Relational Calculus (TRC) 44
     - 2.3.2 Domain Relational Calculus (DRC) 45
   - 2.4 DRC Vs TRC 51
   - 2.5 Relational Algebra Vs Relational Calculus 52

UNIT–II

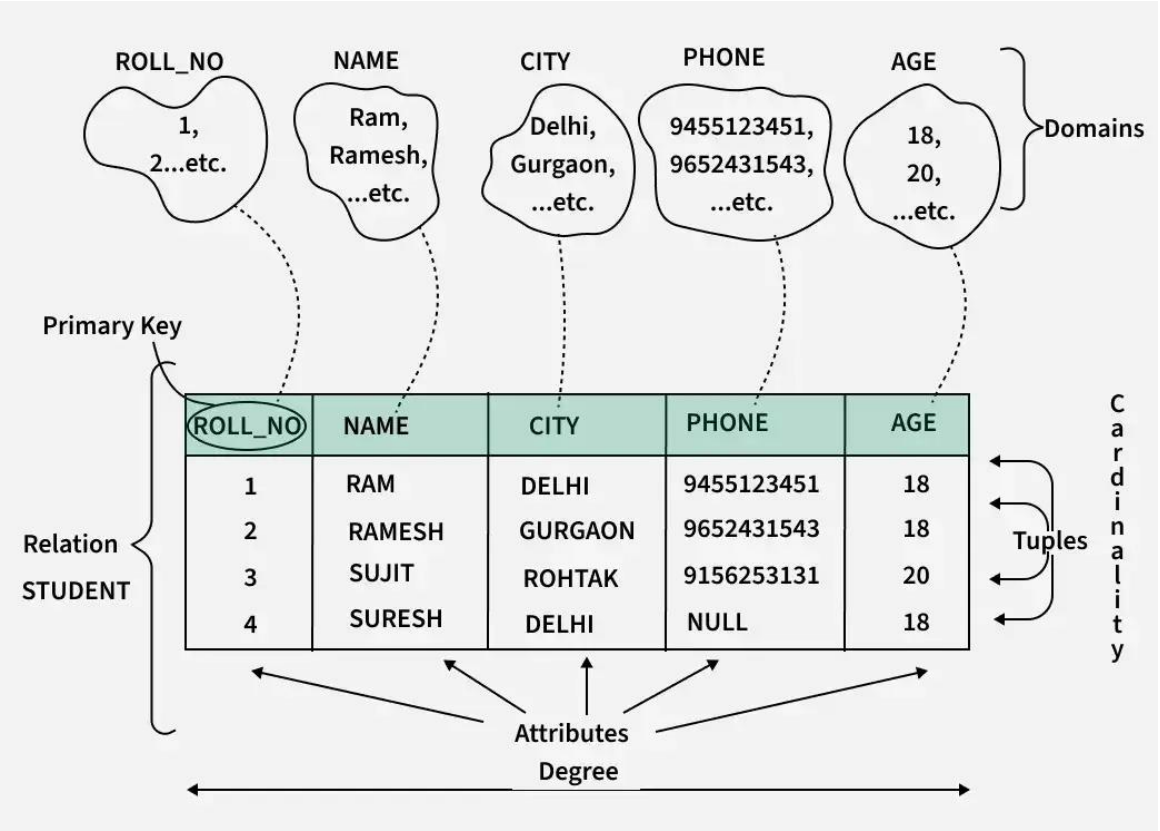
The Relational Model

Introduction to the Relational Model

The Relational Model organizes data using tables (relations) consisting of rows and columns.

- The relational model represents how data is stored and managed in Relational Databases where data is organized into tables, each known as a relation.
- Each row of a table represents an entity or record and each column represents a particular attribute of that entity.
- The relational model transforms conceptual designs from ER diagrams into implementable structures. These structures are used in relational database systems like Oracle SQL and MySQL.

Example: Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE and AGE shown in the table.



Key Terms in the Relational Model

1. **Attribute**: Attributes are the properties that define an entity. For example, ROLL_NO, NAME, ADDRESS etc.
2. **Relation Schema**: A relation schema defines the structure of the relation and represents the name of the relation with its attributes. For example, STUDENT (ROLL_NO, NAME, ADDRESS, PHONE and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation it is called Relational Schema.

3. **Tuple**: A Tuple represents a row in a relation. Each tuple contains a set of attribute values that describe a particular entity. For example, (1, RAM, DELHI, 9455123451, 18) is a tuple in the STUDENT table.
4. **Relation Instance**: The set of tuples of a relation at a particular instance of time is called a relation instance. It can change whenever there is an insertion, deletion or update in the database.
5. **Degree**: The number of attributes in the relation is known as the degree of the relation. For example, The STUDENT relation has a degree of 5, as it has 5 attributes.
6. **Cardinality**: The number of tuples in a relation is known as cardinality. For example, The STUDENT relation defined above has cardinality 4.
7. **NULL Values**: The value which is not known or unavailable is called a NULL value. It is represented by NULL. For example, PHONE of STUDENT having ROLL_NO 4 is NULL.

Types of Keys in the Relational Model

1. **Primary Key**: A Primary Key uniquely identifies each tuple in a relation. It must contain unique values and cannot have NULL values.
2. **Candidate Key**: A Candidate Key is a set of attributes that can uniquely identify a tuple in a relation.
3. **Super Key:** A Super Key is a set of attributes that can uniquely identify a tuple.
4. **Foreign Key**: A Foreign Key is an attribute in one relation that refers to the primary key of another relation.
5. **Composite Key**: A Composite Key is formed by combining two or more attributes to uniquely identify a tuple.

Relational Model Notation

- Relation schema R of degree n is denoted by R(A1, A2, ...,An).
- Uppercase letters Q, R, S denote relation names.
- Lowercase letters q, r, s denote relation states.
- Letters t, u, v denote tuples.
- In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation.
- An attribute A can be qualified with the relation name R to which it belongs by using the dot notation R.A for example, STUDENT.Name or STUDENT.Age.
- An n-tuple t in a relation r(R) is represented as t = <v1, v2,..., vn> where vi is the value corresponding to the attribute Ai. The value vi for attribute Ai in tuple t can be accessed using t[Ai] or t.Ai.

Characteristics of the Relational Model

- **Data Representation:** Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
- **Atomic Values:** Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
- **Unique Keys:** Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
- **Attribute Domain:** Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.
- **Data Independence:** The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
- **Relational Operations**: Supports operations like selection, projection, join, union and intersection, enabling powerful data retrieval and manipulation.
- **Data Consistency:** Ensures data consistency through constraints, reducing redundancy and anomalies.
- **Set-Based Representation:** Tables in the relational model are treated as sets and operations follow mathematical set theory principles.

Constraints in Relational Model

While designing the Relational Model, we define some conditions which must hold for data present in the database are called Constraints. These constraints are checked before performing any operation (insertion, deletion and updation) in the database. If there is a violation of any of the constraints, the operation will fail.

1) **Domain Constraints**

   Ensure that the value of each attribute A in a tuple must be an atomic value derived from its specified domain, dom(A). Domains are defined by the data types associated with the attributes. Common data types include:

   - **Numeric types:** Includes integers (short, regular and long) for whole numbers and real numbers (float, double-precision) for decimal values, allowing precise calculations.
   - **Character types:** Consists of fixed-length (CHAR) and variable-length (VARCHAR, TEXT) strings for storing text data of various sizes.
   - **Boolean values:** Stores true or false values, often used for flags or conditional checks in databases.
   - **Specialized types:** Includes types for date (DATE), time (TIME), timestamp (TIMESTAMP) and money (MONEY), used for precise handling of time-related and financial data.

2) **Key Integrity**

   Every relation in the database should have at least one set of attributes that defines a tuple uniquely. Those set of attributes is called keys. e.g.; ROLL_NO in STUDENT is key. No two students can have the same roll number. So a key has two properties:

   - It should be unique for all tuples.
   - It can't have NULL values.

3) Referential Integrity Constraints

When one attribute of a relation can only take values from another attribute of the same relation or any other relation, it is called referential integrity. Let us suppose we have 2 relations

Table: STUDENT

ROLL_NO	NAME	ADDRESS	PHONE	AGE	BRANCH_CODE
1	RAM	DELHI	9455123451	18	CS
2	RAMESH	GURGAON	9652431543	18	CS
3	SUJIT	ROHTAK	9156253131	20	ECE
4	SURESH	DELHI		18	IT

Table: BRANCH

BRANCH_CODE	BRANCH_NAME
CS	COMPUTER SCIENCE
IT	INFORMATION TECHNOLOGY
ECE	ELECTRONICS AND COMMUNICATION ENGINEERING
CV	CIVIL ENGINEERING

Explanation: BRANCH_CODE of STUDENT can only take the values which are present in BRANCH_CODE of BRANCH which is called referential integrity constraint. The relation which is referencing another relation is called REFERENCING RELATION (STUDENT in this case) and the relation to which other relations refer is called REFERENCED RELATION (BRANCH in this case).

Anomalies in the Relational Model

An anomaly is an irregularity or something which deviates from the expected or normal state. When designing databases, we identify three types of anomalies: Insert, Update, and Delete.

- **Insertion Anomaly in Referencing Relation:** We can't insert a row in REFERENCING RELATION if referencing attribute's value is not present in the referenced attribute value. e.g.; Insertion of a student with BRANCH_CODE 'ME' in STUDENT relation will result in an error because 'ME' is not present in BRANCH_CODE of BRANCH.
- **Deletion/ Updation Anomaly in Referenced Relation:** We can't delete or update a row from REFERENCED RELATION if the value of REFERENCED ATTRIBUTE is used in the value of REFERENCING ATTRIBUTE. For example, If we try to delete a tuple from BRANCH having BRANCH_CODE 'CS', it will result in an error because 'CS' is referenced by BRANCH_CODE of STUDENT, but if we try to delete the row from BRANCH with BRANCH_CODE CV, it will be deleted as the value is not been used by referencing relation.

Codd Rules in Relational Model

E.F. Codd, the creator of the relational model, proposed 12 rules (known as Codd's 12 Rules) that define what constitutes a relational database system. These rules emphasize the importance of data independence, consistency and structure.

Codd's Rules in DBMS

- Rule 1: The Information Rule - All information, whether it is user information or metadata, that is stored in a database must be entered as a value in a cell of a table. It is said that everything within the database is organized in a table layout.
- Rule 2: The Guaranteed Access Rule - Each data element is guaranteed to be accessible logically with a combination of the table name, primary key (row value), and attribute ame (column value).
- Rule 3: Systematic Treatment of NULL Values - Every Null value in a database must be given a systematic and uniform treatment.
- Rule 4: Active Online Catalog Rule - The database catalog, which contains metadata about the database, must be stored and accessed using the same relational database management system.
- Rule 5: The Comprehensive Data Sublanguage Rule - A crucial component of any efficient database system is its ability to offer an easily understandable data manipulation language (DML) that facilitates defining, querying, and modifying information within the database.
- Rule 6: The View Updating Rule - All views that are theoretically updatable must also be updatable by the system.
- Rule 7: High-level Insert, Update, and Delete - A successful database system must possess the feature of facilitating high-level insertions, updates, and deletions that can grant users the ability to conduct these operations with ease through a single query.
- Rule 8: Physical Data Independence - Application programs and activities should remain unaffected when changes are made to the physical storage structures or methods.
- Rule 9: Logical Data Independence - Application programs and activities should remain unaffected when changes are made to the logical structure of the data, such as adding or modifying tables.
- Rule 10: Integrity Independence - Integrity constraints should be specified separately from application programs and stored in the catalog. They should be automatically enforced by the database system.
- Rule 11: Distribution Independence - The distribution of data across multiple locations should be invisible to users, and the database system should handle the distribution transparently.
- Rule 12: Non-Subversion Rule - If the interface of the system is providing access to low-level records, then the interface must not be able to damage the system and bypass security and integrity constraints.

Advantages

- **Simple model:** Relational Model is simple and easy to use in comparison to other languages.
- **Flexible:** Relational Model is more flexible than any other data model present.
- **Secure:** Relational Model is more secure than any other relational model.
- **Data Accuracy:** Data is more accurate in the relational data model.
- **Data Integrity:** The integrity of the data is maintained in the relational model.
- **Operations can be Applied Easily:** It is better to perform operations in the relational model.

Disadvantages

- **Performance:** The relational model can experience performance issues with very large databases.
- **Complexity for Complex Data**: The model struggles with hierarchical or complex data relationships, which might be better handled with other models like the Graph or Document model.
- **Normalization Overhead:** Extensive use of normalization can result in complex queries and slower performance.

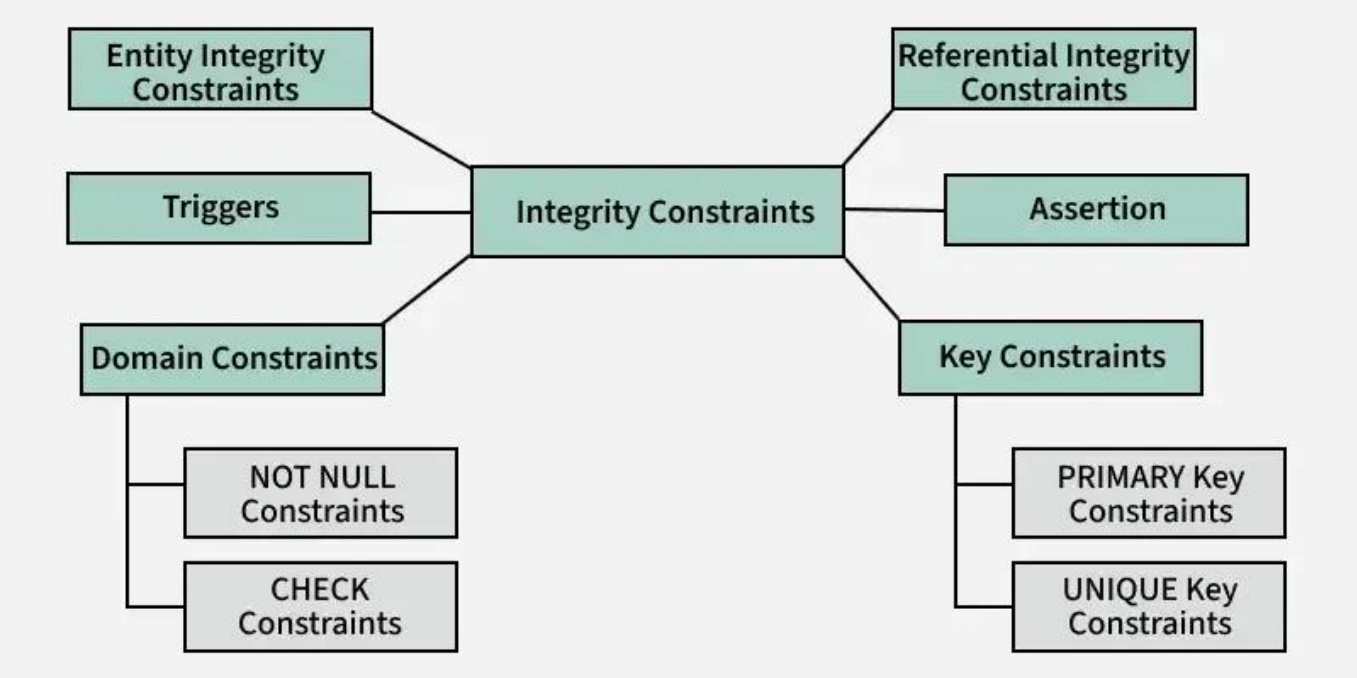
Integrity Constraints

Integrity constraints are a set of rules used in DBMS to ensure that the data in a database is accurate, consistent and reliable. These rules helps in maintaining the quality of data by ensuring that the processes like adding, updating or deleting information do not harm the integrity of the database. Integrity constraints also define how different parts of the database are connected and ensure that these relationships remain valid. They play an essential role in making sure the data is meaningful and follows the logical structure of the database.

Integrity Constraints

Integrity constraints in a Database Management System are rules that help keep the data in a database accurate, consistent and reliable. They act like a set of guidelines that ensure all the information stored in the database follows specific standards.

\



Example: Making sure every customer has a valid email address & ensuring that an order in the database is always linked to an existing customer.

Types of Integrity Constraints

There are Different types of Integrity Constraints used in DBMS, these are:

1) Domain Constraints
2) Entity Integrity Constraints
3) Key Constraints
4) Referential integrity constraints
5) Assertion
6) Triggers

1) Domain Constraints

Domain constraints are a type of integrity constraint that ensure the values stored in a column (or attribute) of a database are valid and within a specific range or domain. In simple terms, they define what type of data is allowed in a column and restrict invalid data entry. The data type of domain include string, char, time, integer, date, currency etc. The value of the attribute must be available in comparable domains.

Example: Below table demonstrates domain constraints in action by enforcing rules for each column.

Student_Id	Name	Semester	Age
21CSE100	Aniket Kumar	6th	20
21CSE101	Shashwat Dubey	7th	21
21CSE102	Manvendra Sharma	8th	22
21CSE103	Ashmit Dubey	5th	20

- **Student_Id:** Must be unique and follow a specific format like 21CSE###. No duplicates or invalid formats allowed.
- **Name**: Accepts only valid text (no numbers) and cannot be left empty (NOT NULL constraint).
- **Semester**: Allows specific values like 5th, 6th, etc., and ensures valid input (e.g., no 10th if not permitted).
- **Age**: Must be an integer within a reasonable range (e.g., 18-30) and cannot contain invalid data like negative numbers or text.

Types of Domain Constraints:

- **NOT NULL Constraint:** Ensures No records can have NULL value.
- **CHECK Constraint:** This Constraint Checks for any specified condition over any attribute.

Importance of Domain Constraints:

- They prevent invalid or inconsistent data from entering the database.
- They ensure the database is reliable and follows predefined business rules.
- They make the database easier to manage and maintain by reducing errors.

Example: Let, the not-null constraint be specified on the "Semester" attribute in the relation/table given below, then the data entry of 4th tuple will violate this integrity constraint, because the "Semester" attribute in this tuple contains null value. To make this database instance a legal instance, its entry must not be allowed by database management system.

Student_id	Name	Semester	Age
21CSE1001	Sonali Rao	5th	20
21CSE1012	Anjali Gupta	5th	21
21CSE1023	Aastha Singh	5th	22
21CSE1034	Ayushi Singh	NULL	20

2) Entity Integrity Constraints

Entity integrity constraints state that primary key can never contain null value because primary key is used to determine individual rows in a relation uniquely, if primary key contains null value then we cannot identify those rows. A table can contain null value in it except primary key field.

Key Features of Entity Integrity Constraints:

- **Uniqueness**: The primary key value must be unique for each row in the table. No duplicate entries are allowed in the primary key column.
- **NOT NULL**: The primary key column cannot contain NULL values, as every row must have a valid identifier.
- **Essential for Table Design**: Ensures that every record in the table can be uniquely identified, preventing ambiguity.

Example: It is not allowed because it is containing primary key (Student_id) as NULL value.

Student_id	Name	Semester	Age
21CSE101	Ramesh	5th	20
21CSE102	Kamlesh	5th	21
21CSE103	Aakash	5th	22
NULL	Mukesh	5th	20

3) Key Constraints

Key constraints ensure that certain columns or combinations of columns in a table uniquely identify each row. These rules are essential for maintaining data integrity and preventing duplicate or ambiguous records.

Why Key Constraints Are Important?

- **Prevent Duplicates:** Ensure unique identification of rows.
- **Maintain Relationships:** Enable proper linking between tables (via foreign keys).
- **Enforce Data Integrity:** Prevent invalid or inconsistent data.

Example: It is now acceptable because all rows must be unique.

Student_id	Name	Semester	Age
21CSE101	Ramesh	5th	20
21CSE102	Kamlesh	5th	21
21CSE103	Aakash	5th	22
21CSE102	Mukesh	5th	20

3.1 Primary Key Constraints

It states that the primary key attributes are required to be unique and not null. That is, primary key attributes of a relation must not have null values and primary key attributes of

two tuples must never be same. This constraint is specified on database schema to the primary key attributes to ensure that no two tuples are same.

Example: Here, in the below example the Student_id is the primary key attribute. The data entry of 4th tuple violates the primary key constraint that is specifies on the database schema and therefore this instance of database is not a legal instance.

Student_id	Name	Semester	Age
101	Ramesh	5th	20
102	Kamlesh	5th	21
103	Akash	5th	22

- **Unique Values:** Each student_id must be unique. 101, 102, 103 are valid. Inserting 101 again would result in an error.
- **Not NULL:** student_id cannot be NULL.
- **Invalid**: A row with NULL for student_id will be rejected.

3.2 Unique Key Constraints

The Unique key constraint in DBMS ensures that all values in a specified column (or group of columns) are distinct across the table. It prevents duplicate entries, maintaining data integrity, but unlike the primary key, it allows one NULL value.

Example: Here, in the below example the Email column has NULL value in 2nd record.

Employee_ID	Email	Name
1	aniket@example.com	Aniket Kumar
2	NULL	Shashwat Dubey
3	shashwat@example.com	Manvendra Sharma

- **Unique Values**: The email column must contain unique values. aniket@example.com and shashwat@example.com are valid. Adding another row with aniket@example.com would result in an error.
- **Allows One NULL**: The email column can contain one NULL value.
- **Valid:** NULL in the second row.
- **Invalid**: Adding another row with NULL in email will be rejected.

4) Referential integrity constraints

Referential integrity constraints are rules that ensure relationships between tables remain consistent. They enforce that a foreign key in one table must either match a value in the referenced primary key of another table or be NULL. This guarantees the logical connection between related tables in a relational database.

Why Referential Integrity Constraints Are Important?

- Maintains Consistency: Ensures relationships between tables are valid.
- Prevents Orphan Records: Avoids cases where a record in a child table references a non-existent parent record.

- Enforces Logical Relationships: Strengthens the logical structure of a relational database.

Example: Here, in below example Block_No 22 entry is not allowed because it is not present in 2nd table.

Student_id	Name	Semester	Block_No
22CSE101	Ramesh	5th	20
21CSE105	Kamlesh	6th	21
22CSE102	Aakash	5th	20

Block_No	Block Location
20	Chandigarh
21	Punjab
25	Delhi

5) Assertion

An assertion is a declarative mechanism in a database that ensures a specific condition or rule is always satisfied across the entire database. It is a global integrity constraint, meaning it applies to multiple tables or the entire database rather than being limited to a single table or column. An assertion in SQL-92 takes the form:

create assertion <assertion-name> check <predicate>

When an assertion is made, the system tests it for validity. This testing may introduce a significant amount of overhead; hence assertions should be used with great care.

Example of an Assertion:

```
CREATE ASSERTION sum_constraint
CHECK (
NOT EXISTS (
SELECT *
FROM branch
WHERE (
SELECT SUM(amount)
FROM loan
WHERE loan.branch_name = branch.branch_name
) >= (
SELECT SUM(amount)
FROM account
WHERE account.branch_name = branch.branch_name
)
)
);
```

Explanation:

The following SQL statement creates an assertion to ensure that the total loan amount at each branch is always less than the total account balances at the same branch.

- **Purpose**: This assertion enforces a global business rule. The sum of all loan amounts for a branch must always be less than the sum of all account balances in the same branch. This prevents branches from issuing loans beyond their financial capacity.
- **Subqueries**: The first subquery (SELECT SUM(amount) FROM loan) calculates the total loan amount for each branch. The second subquery (SELECT SUM(amount) FROM account) calculates the total balance of accounts for the same branch.
- **Condition**: The NOT EXISTS clause ensures there is no branch where the loan amount is greater than or equal to the account balance.
- **Behavior**: If a transaction (e.g., inserting a loan or updating an account) violates this rule, the operation will be rejected by the database.

6) Triggers

A trigger is a procedural statement in a database that is automatically executed in response to certain events such as INSERT, UPDATE, or DELETE. Triggers are often used to enforce complex integrity constraints or implement business rules that cannot be captured using standard constraints like primary keys or foreign keys.

Example SQL Trigger:

```
CREATE TRIGGER handle_overdraft
AFTER UPDATE ON account
FOR EACH ROW
BEGIN
-- Check if the balance has become negative after the update
IF NEW.balance < 0 THEN
-- Set the account balance to zero
UPDATE account
SET balance = 0
WHERE account_number = NEW.account_number;
-- Create a loan record with the same account number as the loan number
INSERT INTO loan (loan_number, loan_amount)
VALUES (NEW.account_number, ABS(NEW.balance)); -- ABS to ensure positive loan amount
END IF;
END;
```

Enforcing Integrity Constraints

Integrity Constraints are specified when a relation is created and enforced when a relation is modified. The impact of domain, PRIMARY KEY, and UNIQUE constraints is straightforward: If an insert, delete, or update command causes a violation, it is rejected. Every potential Integrity violation is generally checked at the end of each SQL statement execution, although it can be deferred until the end of the transaction executing the statement.

On the other hand, insertions of Students tuples do not violate referential integrity, and deletions of Students tuples could cause violations.

SQL provides several alternative ways to handle foreign key violations. We must consider three basic questions:

1) What should we do if an Enrolled row is inserted, with a student id column value that does not appear in any row of the Students table?
In this case, the INSERT command is simply rejected.

2) What should we do if a Students row is deleted?
The options are:

- ⇨ Delete all Enrolled rows that refer to the deleted Students row.
  Syntax in Mysql

```
CREATE TABLE table_name
(
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....
    FOREIGN KEY (column) REFERENCES table_name1(column) ON DELETE CASCADE
);
```

- ⇨ Disallow the deletion of the Students row if an Enrolled row refers to it.

  Syntax in Mysql

```
CREATE TABLE table_name
(
        column1 datatype,
        column2 datatype,
        column3 datatype,
        ....
        FOREIGN KEY (column) REFERENCES table_name1(column) ON DELETE NO ACTION
);
```

⇨ Set the studid column to the sid of some (existing) 'default' student, for every Enrolled row that refers to the deleted Students row.

Syntax in Mysql

```
CREATE TABLE table_name
(
column1 datatype,
column2 datatype,
column3 datatype,
....
FOREIGN KEY (column) REFERENCES table_name1(column) ON DELETE SET DEFAULT
);
```

3) What should we do if the primary key value of a Students row is updated?
The options here are similar to the previous case. Replace ON DELETE with ON UPDATE

Syntax in Mysql

```
CREATE TABLE table_name
(
    column1 datatype,
    column2 datatype,
    column3 datatype,
    ....
    FOREIGN KEY (column) REFERENCES table_name1(column) ON UPDATE NO ACTION
);
```

Querying Relational Data

SQL has its own querying methods to interact with the database. But how do these queries work in the database? These queries work similarly to Relational Algebra that we study in mathematics. In the database, we have tables participating in relational Algebra.

Relational Database systems are expected to be equipped with a query language that assists users to query the database. Relational Query Language is used by the user to communicate with the database user requests for the information from the database. Relational algebra breaks the user requests and instructs the DBMS to execute the requests. It is the language by which the user communicates with the database. They are generally on a higher level than any other programming language. These relational query languages can be Procedural and Non-Procedural.

Types of Relational Query Language

There are two types of relational query language:

1) Procedural Query Language
2) Non-Procedural Language

1) Procedural Query Language

In Procedural Language, the user instructs the system to perform a series of operations on the database to produce the desired results. Users tell what data to be retrieved from the database and how to retrieve it. Procedural Query Language performs a set of queries instructing the DBMS to perform various transactions in sequence to meet user requests.

Relational Algebra is a Procedural Query Language

Relational Algebra could be defined as the set of operations on relations.

There are a few operators that are used in relational algebra -

- **Select (sigma):** Returns rows of the input relation that satisfy the provided predicate. It is unary Operator means requires only one operand.
- **Projection (π):** Show the list of those attribute which we desire to appear and rest other attributes are eliminated from the table. It seperates the table vertically.
- **Set Difference (-):** It returns the difference between two relations. If we have two relations R and S them R-S will return all the tuples (row) which are in relation R but not in Relation S, It is binary operator.
- **Cartesian Product (X):** Combines every tuple (row) of one table with every tuple (row) in other table, also referred as cross Product. It is a binary operator.
- **Union (U):** Outputs the union of tuples from both the relations. Duplicate tuples are eliminated automatically. It is a binary operator means it require two operands.

2) Non-Procedural Language

In Non-Procedural Language user outlines the desired information without giving a specific procedure or without telling the steps by step process for attaining the information. It only gives a single Query on one or more tables to get .The user tells what is to be retrieved from the database but does not tell how to accomplish it.

For Example: get the name and the contact number of the student with a Particular ID will have a single query on STUDENT table.

Relational Calculus exists in two forms:

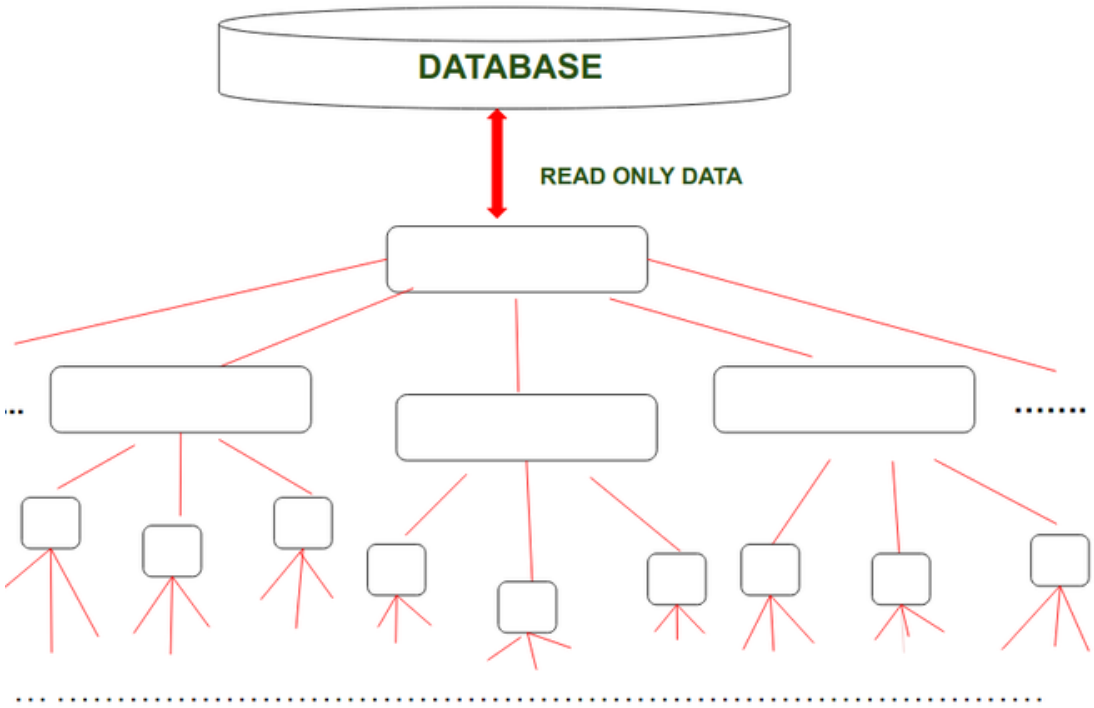
- **Tuple Relational Calculus (TRC):** Tuple Relational Calculus is a non-procedural query language, It is used for selecting the tuples that satisfy the given condition or predicate. The result of the relation can have one or more tuples (row).
- **Domain Relational Calculus (DRC):** Domain Relational Calculus is a Non Procedural Query Language, the records are filtered based on the domains, DRC uses the list of attributes to be selected from relational based on the condition.

Logical Database Design

A Logical Database is a special type of ABAP (Advance Business Application and Programming) that is used to retrieve data from various tables and the data is interrelated to each other. Also, a logical database provides a read-only view of Data.

Structure of Logical Database:

A Logical database uses only a hierarchical structure of tables i.e. Data is organized in a Tree-like Structure and the data is stored as records that are connected to each other through edges (Links). Logical Database contains Open SQL statements which are used to read data from the database. The logical database reads the program, stores them in the program if required, and passes them line by line to the application program.



Features of Logical Database:

In this section, let us look at some features of a logical database:

- We can select only that type of Data that we need.
- Data Authentication is done in order to maintain security.
- Logical Database uses hierarchical Structure due to this data integrity is maintained.

Goal of Logical Database:

The goal of Logical Database is to create well-structured tables that reflect the need of the user. The tables of the Logical database store data in a non-redundant manner and foreign keys will be used in tables so that relationships among tables and entities will be supported.

Tasks of Logical Database:

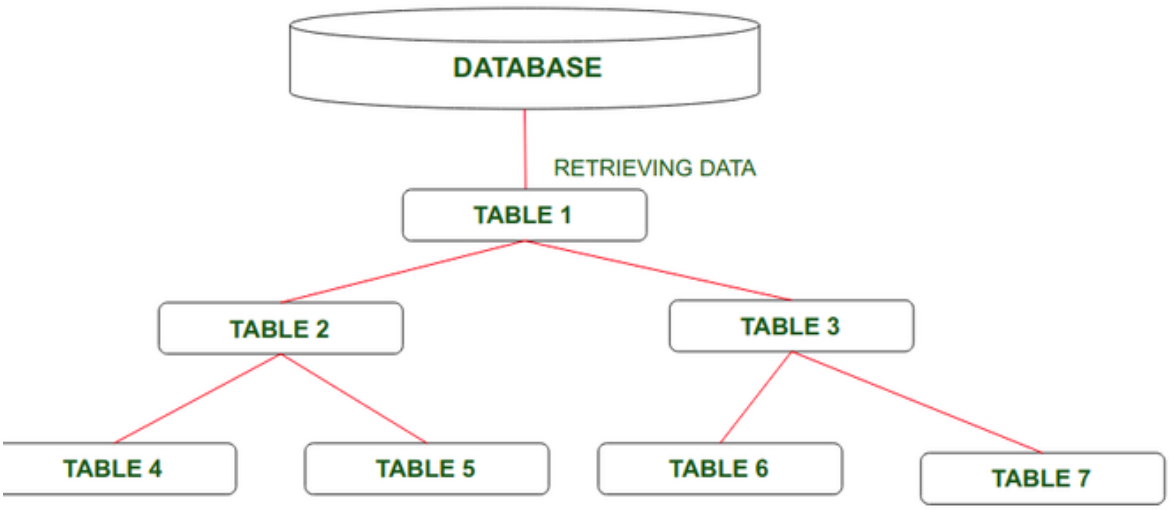
- Below is some important task of Logical Database:
- With the help of the Logical database, we will read the same data from multiple programs.
- A logical database defines the same user interface for multiple programs.
- Logical Database ensures the Authorization checks for the centralized sensitive database.
- With the help of a Logical Database, Performance is improved. Like in Logical Database we will use joins instead of multiple SELECT statements, which will improve response time and this will increase the Performance of Logical Database.

Data View of Logical Database:

Logical Database provides a particular view of Logical Database tables. A logical database is appropriately used when the structure of the Database is Large. It is convenient to use flow i.e

- SELECT
- READ
- PROCESS
- DISPLAY

In order to work with databases efficiently. The data of the Logical Database is hierarchical in nature. The tables are linked to each other in a Foreign Key relationship.
Diagrammatically, the Data View of Logical Database is shown as:



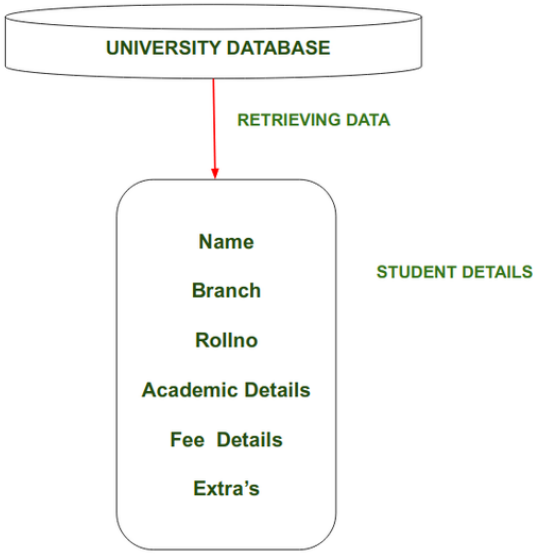
Points To Remember:

- Tables must have Foreign Key Relationship.
- A logical Database consists of logically related tables that are arranged in a hierarchical manner used for reading or retrieving Data.
- Logical Database consist of three main elements:
  - Structure of Database

- Selections of Data from Database
- Database Program

- If we want to improve the access time on data, then we use VIEWS in Logical Database.

Example: Suppose in a University or College, a HOD wants to get information about a specific student. So for that, he firstly retrieves the data about its batch and Branch from a large amount of Data, and he will easily get information about the required Student but didn't alter the information about it.



Advantages of Logical Database:

- In a Logical database, we can select meaningful data from a large amount of data.
- Logical Database consists of Central Authorization which checks for Database Accesses is Authenticated or not.
- In this Coding, the part is less required to retrieve data from the database as compared to Other Databases.
- Access performance of reading data from the hierarchical structure of the Database is good.
- Easy to understand user interfaces.
- Logical Database firstly check functions which further check that user input is complete, correct, and plausible.

Disadvantages of Logical Database:

- Logical Database takes more time when the required data is at the last because if that table which is required at the lowest level then firstly all upper-level tables should be read which takes more time and this slows down the performance.
- In Logical Database ENDGET command doesn't exist due to this the code block associated with an event ends with the next event statement.

Introduction to Views

A view is a table whose rows are not explicitly stored, a view is a virtual table based on the result-set of an SQL statement. A view can contain all rows of a table or select rows from a table. A view can be created from one or many tables which depends on the written SQL query to create a view.

A view is generated to show the information that the end-user requests the data according to specified needs rather than complete information of the table.

Use of Views

- **Data Security**: Views restrict access to specific data. Instead of giving users full access to an entire sensitive table, you can create a view that hides confidential columns (like salaries).
- **Simplified Queries**: They hide the complexity of multiple joins, subqueries, and calculations. Users can simply query the view as if it were a single, clean table.
- **Data Consistency:** You ensure that complex business logic, filters, and aggregations are applied uniformly, preventing users from writing redundant or incorrect queries.

Common Use Cases

- Report Generation: Creating a pre-aggregated view to track monthly sales or employee performance, which can be queried directly by reporting tools.
- Backward Compatibility: If a database schema changes, a view can be created to mimic the old table structure, ensuring existing applications don't break.

Basic Operations - Views are created, modified, and dropped using standard SQL commands: [1]

- **Create a View:** Use the CREATE VIEW statement followed by a SELECT query.
- **Query a View:** You can pull data from a view exactly as you would a normal table using SELECT * FROM view_name.
- **Update a View:** The CREATE OR REPLACE VIEW command modifies an existing view without having to drop and recreate it.
- **Delete a View:** Use the DROP VIEW command to remove it from the database. [1, 2, 3]

A SQL View is a virtual table created from the result of a SELECT query. It does not store data physically but displays data stored in underlying tables. Views help simplify complex queries, enhance security, and present data in a cleaner, customized format.

Example: First, we will create a demo SQL database and table, on which we will use the TRUNCATE TABLE command.

S_ID	NAME	ADDRESS
1	Liam	New York
2	Sophia	London
3	Noah	Toronto

Query:

```
CREATE VIEW StudentView AS
SELECT NAME, ADDRESS
FROM StudentDetails;
SELECT * FROM StudentView;
```

Output:

NAME	ADDRESS
Liam	New York
Sophia	London
Noah	Toronto

CREATE VIEWS in SQL

We can create a view using CREATE VIEW statement. A View can be created from a single table or multiple tables.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_nameWHERE condition;
```

- **view_name**: Name for the View
- **table_name**: Name of the table
- **condition**: Condition to select rows

Example 1: Creating a Simple View from a Single Table

First, we will create a demo SQL database and table, on which we will use the View command.

S_ID	NAME	ADDRESS
1	Liam	London
2	Sofia	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome

In this example, we create a View named DetailsView from the table StudentDetails.

Query:

```
CREATE VIEW DetailsView AS
SELECT NAME, ADDRESS
FROM StudentDetails
WHERE S_ID < 5;
```

Use the below query to retrieve the data from this view

```
SELECT * FROM DetailsView;
```

Output:

NAME	ADDRESS
Liam	London
Sofia	Berlin
Mateo	Madrid
Emma	Paris

Example 2: Creating a View From Multiple Tables

In this example, we create a View MarksView that combines data from bothtables StudentDetails and StudentMarks. To create a View from multiple tables we can simply include multiple tables in the SELECT statement.

NAME	MARKS
Liam	85
Sofia	92
Mateo	78
Emma	88

Query:

```
CREATE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

To display data of View MarksView:

```
SELECT * FROM MarksView;
```

Output:

NAME	ADDRESS	MARKS
Liam	London	85
Sofia	Berlin	92
Mateo	Madrid	78
Emma	Paris	88

Managing Views

Here are some common operations used to manage views in SQL:

2) Listing all Views in a Database

We can list all the views in a database using the SHOW FULL TABLES statement or by querying the information_schema tables.

```
USE "database_name";
SHOW FULL TABLES WHERE table_type LIKE "%VIEW";
```

Using information_schema

```
SELECT table_name
FROM information_schema.views
WHERE table_schema = 'database_name';
```

OR

```
SELECT table_schema, table_name, view_definition
FROM information_schema.views
WHERE table_schema = 'database_name';
```

3) Deleting a View

SQL allows us to delete an existing View. We can delete or drop View using the DROP statement.

Syntax:

```
DROP VIEW view_name;
```

Example: In this example, we are deleting the View MarksView.

```
DROP VIEW MarksView;
```

4) Updating a View Definition

If we want to update the existing data within the view, use the **UPDATE** statement.

```
UPDATE view_name
SET column1 = value1, column2 = value2...., columnN = valueN
WHERE [condition];
```

If you want to update the view definition without affecting the data, use the CREATE OR REPLACE VIEW statement. For example, let's add the Age column to the MarksView:

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Note: Not all views can be updated using the UPDATE statement.

Rules to Update Views in SQL:

Certain conditions need to be satisfied to update a view. If any of these conditions are **not** met, the view can not be updated.

- The SELECT statement which is used to create the view should not include GROUP BY clause or ORDER BY clause.
- The SELECT statement should not have the DISTINCT keyword.
- The View should have all NOT NULL values.
- The view should not be created using nested queries or complex queries.
- The view should be created from a single table. If the view is created using multiple tables then we will not be allowed to update the view.

Advanced Techniques with Views

Here are some advanced ways to work with SQL views:

1) Updating Data Through Views

We can use the CREATE OR REPLACE VIEW statement to add or replace fields from a view If we want to update the view MarksView and add the field AGE to this View from StudentMarks Table, we can do this by:

Example:

```
CREATE OR REPLACE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS,
StudentMarks.AGE
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;
```

If we fetch all the data from MarksView now as:

```
SELECT * FROM MarksView;
```

Output:

NAME	S_ID	ADDRESS	MARKS
Liam	1	London	85
Sofia	2	Berlin	92
Mateo	3	Madrid	78
Emma	4	Paris	88

2) Inserting Data into Views

We can insert a row in a View in the same way as we do in a table. We can use the INSERT INTO statement of SQL to insert a row in a View. In the below example, we will insert a new row in the View StudentDetailswhich we have created above in the example of "creating views from a single table".

Example:

```
INSERT INTO StudentDetails(NAME, ADDRESS)
VALUES("John","German");
```

If we fetch all the data from DetailsView now as,

```
SELECT * FROM StudentDetails;
```

Output:

NAME	ADDRESS
Liam	London
Sofia	Berlin
Mateo	Madrid
Emma	Paris
Noah	Rome
John	German

3) Deleting a row from a View

Deleting rows from a view works the same as deleting from a table. Using the DELETE command removes the row from the base table, and the change automatically appears in the view. In this example, we delete the last row from StudentDetails added earlier.

Query:

```
DELETE FROM StudentDetails
WHERE NAME="John";
If we fetch all the data from DetailsView now as,
SELECT * FROM StudentDetails;
```

Output:

NAME	ADDRESS
Liam	London
Sofia	Berlin
Mateo	Madrid
Emma	Paris
Noah	Rome

4) WITH CHECK OPTION Clause

The WITH CHECK OPTION clause ensures that any INSERT or UPDATE on an updatable view must satisfy the view's WHERE condition. If the condition is violated, SQL returns an error. In this example, a view is created with WITH CHECK OPTION to restrict changes that fall outside the defined criteria.

Query:

```
CREATE VIEW SampleView AS
SELECT S_ID, NAME
FROM  StudentDetails
WHERE NAME IS NOT NULL
WITH CHECK OPTION;
UPDATE SampleView SET NAME = 'Mark' WHERE S_ID = 2;
```

Output:

S_ID	NAME	ADDRESS
1	Liam	London
2	Mark	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome

Destroying/Altering Tables and Views

Altering Table

The SQL ALTER TABLE statement is used to modify an existing table's structure without deleting it. It helps update the design of a database as requirements change.

- Can add, delete or modify columns in a table.
- Can also rename a table or change data types and constraints.
- Useful for adjusting database structure without losing data.

Example: First, we will create a demo SQL database and Employees table, on which we will use the ALTER TABLE command.

Employees

ID	Name	Department	Salary
1	John	HR	50000
2	Emma	IT	60000

Employees Table

Query:

```
ALTER TABLE Employees RENAME TO Staff;
```

Output: Output after renaming table

Staff

ID	Name	Department	Salary
1	John	HR	50000
2	Emma	IT	60000

Syntax:

```
ALTER TABLE table_name [ADD | DROP | MODIFY] column_name datatype;
```

- **table_name:** name of the table you want to modify.
- **ADD:** used to add a new column.
- **DROP:** used to remove an existing column.
- **MODIFY:** used to change datatype or definition of an existing column.

Use Cases for SQL ALTER TABLE

1) **ADD -** The ADD clause is used to add a new column to an existing table. You must specify the name of the new column and its data type.

Syntax:

ALTER TABLE table_name
ADD column_name datatype;

Query:

ALTER TABLE Staff
ADD Email VARCHAR(100);

Output:

Staff

ID	Name	Department	Salary	Email
1	John	HR	50000	
2	Emma	IT	60000	

Output after adding column

Here, we are adding a column named Email to Stafftable

2) MODIFY - The MODIFY (or ALTER COLUMN in some databases like SQL Server) clause is used to modify the definition of an existing column, such as changing its data type or size.

Syntax:

ALTER TABLE table_name
MODIFY COLUMN column_name datatype;

Query:

ALTER TABLE Staff
MODIFY COLUMN Address VARCHAR(100);

Here, we are modifying the column named Address datatype that is VARCHAR(100).

3) DROP - The DROP clause allows you to remove a column from a table. Be cautious when using this command as it will permanently remove the column and its data.

Syntax:

ALTER TABLE table_name
DROP COLUMN column_name;

Query:

ALTER TABLE Staff
DROP COLUMN Grade;

Here, we are removing a column named Grade from Staff table

4) RENAME COLUMN - We can rename an existing column using RENAME COLUMN clause. This allows you to change the name of a column while preserving its data type and content.

Syntax:

ALTER TABLE table_name
RENAME COLUMN old_name TO new_name;

Query:

```
ALTER TABLE Staff
RENAME COLUMN Name TO EmployeeName;
```

Output:

Staff

ID	EmployeeName	Department	Salary	Email
1	John	HR	50000	
2	Emma	IT	60000	

Output after renaming column

This renames the column Name to EmployeeName in the Staff table.

5) **RENAME TO -** We can rename an entire table using the RENAME TO clause. This changes the name of the table while preserving its structure and data.

Syntax:

```
ALTER TABLE table_name
RENAME TO new_table_name;
```

Query:

```
ALTER TABLE Staff
RENAME TO Employees;
```

Output:

Employees

ID	EmployeeName	Department	Salary
1	John	HR	50000
2	Emma	IT	60000

Output after renaming table name

This renames the table from Staff to Employees.

Drop Table

The DROP TABLE command permanently removes a table from the database.

- Deletes all rows and dependent objects, including indexes, constraints and triggers.
- Recovery is possible only through a backup or database recovery mechanism.

Syntax:

```
DROP TABLE table_name;
```

The following example demonstrates how to use the DROP TABLE command.

Step1: Create a Database and Table

First, we will create a database and table on which the SQL queries will be run.

```
CREATE DATABASE NewCafe;
USE NewCafe;

CREATE TABLE categories (
   CategoryID INT NOT NULL PRIMARY KEY,
   CategoryName NVARCHAR(50) NOT NULL,
   ItemDescription NVARCHAR(50) NOT NULL
);

INSERT INTO categories (CategoryID, CategoryName, ItemDescription)
VALUES
(1, 'Beverages', 'Soft Drinks'),
(2, 'Condiments', 'Sweet and Savory Sauces'),
(3, 'Confections', 'Sweet Breads');

SELECT * FROM categories;
```

Output:

CategoryID	CategoryName	ItemDescription
1	Beverages	Soft Drinks
2	Condiments	Sweet and Savory Sauces
3	Confections	Sweet Breads

Categories Table

At this point, the categories table has been created with three rows of sample data.

Step2: Drop the Table

Now, let's use the DROP TABLE statement to delete the categories table permanently

Query:

```
DROP TABLE categories;
```

Output:

SQL query successfully executed. However, the result set is empty.

Drop Categories Table

Working of SQL DROP TABLE

The DROP TABLE statement permanently removes a table from the database along with all its associated data and objects.

- Deletes the table, including its data, indexes, triggers, constraints and permissions.
- Once a table is dropped, it cannot be recovered, so this command should be used with caution.
- Using DROP TABLE IF EXISTS prevents errors by ensuring the table is dropped only if it exists.

```
DROP TABLE IF EXISTS categories;
```

- When a partitioned table is dropped, the table definition, all partitions, their data and partition metadata are removed. The partitioning scheme is also deleted if it is not used by other tables.
- Temporary tables can be dropped using the TEMPORARY keyword.

```
DROP TEMPORARY TABLE temp_table_name;
```

- To verify whether a table has been dropped, use commands like SHOW TABLES (MySQL) or query INFORMATION_SCHEMA.TABLES. If the table no longer exists, DESCRIBE or DESC will return an error.

```
SHOW TABLES;
```

Views

A SQL View is a virtual table created from the result of a SELECT query. It does not store data physically but displays data stored in underlying tables. Views help simplify complex queries, enhance security and present data in a cleaner, customized format.

Example: First, we will create a demo SQL database and table, on which we will use the TRUNCATE TABLE command.

S_ID	NAME	ADDRESS
1	Liam	New York
2	Sophia	London
3	Noah	Toronto

Query:

```
CREATE VIEW StudentView AS
SELECT NAME, ADDRESS
FROM StudentDetails;
SELECT * FROM StudentView;
```

Output:

NAME	ADDRESS
Liam	New York
Sophia	London
Noah	Toronto

⇨ **CREATE VIEWS -** We can create a view using CREATE VIEW statement. A View can be created from a single table or multiple tables.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2.....
FROM table_name
WHERE condition;
```

- **view_name**: Name for the View
- **table_name**: Name of the table
- **condition**: Condition to select rows

Example 1: Creating a Simple View from a Single Table

First, create a sample table. Then create a view using the CREATE VIEW statement.

S_ID	NAME	ADDRESS
1	Liam	London
2	Sofia	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome

In this example, we create a View named DetailsViewfrom the table StudentDetails.

Query:

```
CREATE VIEW DetailsView AS
SELECT NAME, ADDRESS
FROM StudentDetails
WHERE S_ID < 5;

Use the below query to retrieve the data from this view

SELECT * FROM DetailsView;
```

Output:

NAME	ADDRESS
Liam	London
Sofia	Berlin
Mateo	Madrid
Emma	Paris

Example 2: Creating a View From Multiple Tables

In this example, we create a View MarksView that combines data from boththables StudentDetails and StudentMarks. To create a View from multiple tables we can simply include multiple tables in the SELECT statement.

NAME	MARKS
Liam	85
Sofia	92
Mateo	78
Emma	88

Query:

```
CREATE VIEW MarksView AS
SELECT StudentDetails.NAME, StudentDetails.ADDRESS, StudentMarks.MARKS
FROM StudentDetails, StudentMarks
WHERE StudentDetails.NAME = StudentMarks.NAME;

To display data of View MarksView:
SELECT * FROM MarksView;
```

Output:

NAME	ADDRESS	MARKS
Liam	London	85
Sofia	Berlin	92
Mateo	Madrid	78
Emma	Paris	88

Managing Views

Here are some common operations used to manage views in SQL:

1) Listing all Views in a Database - We can list all the views in a database using the SHOW FULL TABLES statement or by querying the information_schema tables.

```
USE "database_name";
SHOW FULL TABLES WHERE table_type LIKE "%VIEW";
```

Using information_schema

```
SELECT table_name
FROM information_schema.views
WHERE table_schema = 'database_name';

OR

SELECT table_schema, table_name, view_definition
FROM information_schema.views
WHERE table_schema = 'database_name';
```

2) Deleting a View - SQL allows us to delete an existing View. We can delete or drop View using the DROP statement.

Syntax:

```
DROP VIEW view_name;
```

Example: In this example, we are deleting the View MarksView.

```
DROP VIEW MarksView;
```

3) Updating a View Definition - If we want to update the existing data within the view, use the UPDATE statement.

```
UPDATE view_name
SET column1 = value1, column2 = value2...., columnN = valueN
WHERE [condition];
```

If you want to update the view definition without affecting the data, use the CREATE OR REPLACE VIEW statement. For example, let's add the Age column to the MarksView:

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Note: Not all views can be updated using the UPDATE statement.

Rules to Update Views in SQL

- Certain conditions need to be satisfied to update a view. If any of these conditions are **not** met, the view can not be updated.

- The SELECT statement which is used to create the view should not include GROUP BY clause or ORDER BY clause.
- The SELECT statement should not have the DISTINCT keyword.
- The View should have all NOT NULL values.
- The view should not be created using nested queries or complex queries.
- The view should be created from a single table. If the view is created using multiple tables then we will not be allowed to update the view.

Advanced Techniques with Views

Here are some advanced ways to work with SQL views:

1) Updating Data Through Views

We can use the CREATE OR REPLACE VIEW statement to add or replace fields from a view If we want to update the view MarksView and add the field AGE to this View from StudentDetails Table, we can do this by:

Example:

```
UPDATE MarkView
SET ADDRESS = 'Perth'
WHERE NAME = 'Liam';
```

If we fetch all the data from MarksView now as:

```
SELECT * FROM MarksView;
```

Output:

NAME	S_ID	ADDRESS	MARKS
Liam	1	Perth	85
Sofia	2	Berlin	92
Mateo	3	Madrid	78
Emma	4	Paris	88

2) Inserting Data into Views

We can insert a row in a View in the same way as we do in a table. We can use the INSERT INTO statement of SQL to insert a row in a View. In the below example, we will insert a new row in the View Mawhich we have created above in the example of "creating views from a single table".

Example:

```
INSERT INTO MarkView(NAME, ADDRESS)
VALUES("John","German");
```

If we fetch all the data from DetailsView now as,

```
SELECT * FROM MarkView;
```

Output:

NAME	S_ID	ADDRESS	MARKS
Liam	1	Perth	85
Sofia	2	Berlin	92
Mateo	3	Madrid	78
Emma	4	Paris	88
John		Germany	

Markvoiew

3) Deleting a row from a View

Deleting rows from a view works the same as deleting from a table. Using the DELETE command removes the row from the base table and the change automatically appears in the view. In this example, we delete the last row fromMarkView added earlier.

Query:

```
DELETE FROM MarkView
WHERE NAME="John";
```

If we fetch all the data from MarkView now as,

```
SELECT * FROM MarkView;
```

Output:

NAME	S_ID	ADDRESS	MARKS
Liam	1	Perth	85
Sofia	2	Berlin	92
Mateo	3	Madrid	78
Emma	4	Paris	88

4) WITH CHECK OPTION Clause

The WITH CHECK OPTION clause ensures that any INSERT or UPDATE on an updatable view must satisfy the view's WHERE condition. If the condition is violated, SQL returns an error. In this example, a view is created with WITH CHECK OPTION to restrict changes that fall outside the defined criteria.

Query:

```
CREATE VIEW SampleView AS
SELECT S_ID, NAME
FROM  StudentDetails
WHERE NAME IS NOT NULL
WITH CHECK OPTION;
```

UPDATE SampleView SET NAME = 'Mark' WHERE S_ID = 2;

Output:

S_ID	NAME	ADDRESS
1	Liam	London
2	Mark	Berlin
3	Mateo	Madrid
4	Emma	Paris
5	Noah	Rome

The update succeeds only if the modified row still satisfies the view condition (NAME IS NOT NULL).

Relational Algebra and Calculus

Introduction

Relational Algebra is a formal language used to query and manipulate relational databases, consisting of a set of operations like selection, projection, union, and join. It provides a mathematical framework for querying databases, ensuring efficient data retrieval and manipulation.

- Relational algebra serves as the mathematical foundation for SQL query.
- SQL queries are based on relational algebra operations, enabling users to retrieve data effectively.

Note: Relational algebra simplifies the process of querying databases and makes it easier to understand and optimize query execution for better performance. It is essential for learning SQL

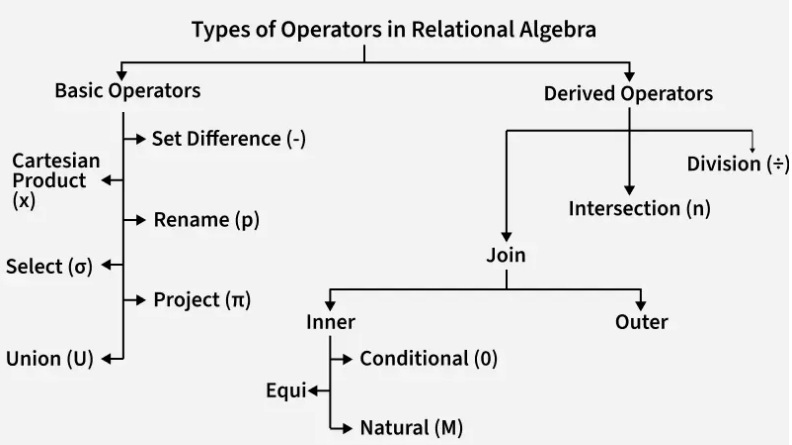
Key Concepts in Relational Algebra

Before explaining relational algebra operations, let's define some fundamental concepts:

- **Relations**: In relational algebra, a relation is a table that consists of rows and columns, representing data in a structured format. Each relation has a unique name and is made up of tuples.
- **Tuples**: A tuple is a single row in a relation, which contains a set of values for each attribute. It represents a single data entry or record in a relational table.
- **Attributes**: Attributes are the columns in a relation, each representing a specific characteristic or property of the data. For example, in a "Students" relation, attributes could be "Name", "Age", and "Grade".
- **Domains**: A domain is the set of possible values that an attribute can have. It defines the type of data that can be stored in each column of a relation, such as integers, strings, or dates.

Basic Operators in Relational Algebra

Relational algebra consists of various basic operators that help us to fetch and manipulate data from relational tables in the database to perform certain operations on relational data. Basic operators are fundamental operations that include selection (σ), projection (π), union (U), set difference (−), Cartesian product (×), and rename (ρ).



Basic Operations

1) Selection(σ)

The Selection Operation is basically used to filter out rows from a given table based on certain given condition. It basically allows us to retrieve only those rows that match the condition as per condition passed during SQL Query.

Example: If we have a relation R with attributes A, B, and C, and we want to select tuples where C > 3, we write:

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

σ(c>3)(R) *will select the tuples which have c more than 3.*

Output:

A	B	C
1	2	4
4	3	4

Explanation: The selection operation only filters rows but does not display or change their order. The projection operator is used for displaying specific columns.

2) Projection(π)

While Selection operation works on rows, similarly projection operation of relational algebra works on columns. It basically allows us to pick specific columns from a given relational table based on the given condition and ignoring all the other remaining columns.

Example: Suppose we want columns B and C from Relation R.

π(B,C)(R) will show following columns.

Output:

B	C
2	4
2	3
3	4

Explanation: By Default, projection operation removes duplicate values.

3) Union(U)

The Union Operator is basically used to combine the results of two queries into a single result. The only condition is that both queries must return same number of columns with same data types. Union operation in relational algebra is the same as union operation in set theory.

Example: Consider the following table of Students having different optional subjects in their course.

FRENCH

Student_Name	Roll_Number
Ram	01
Mohan	02
Vivek	13
Geeta	17

GERMAN

Student_Name	Roll_Number
Vivek	13
Geeta	17
Shyam	21
Rohan	25

If FRENCH and GERMAN relations represent student names in two subjects, we can combine their student names as follows:

π(Student_Name)(FRENCH) U π(Student_Name)(GERMAN)

Output:

Student_Name
Ram
Mohan
Vivek
Geeta
Shyam
Rohan

Explanation: The only constraint in the union of two relations is that both relations must have the same set of Attributes.

4) Set Difference(-)

Set difference basically provides the rows that are present in one table, but not in another tables. Set Difference in relational algebra is the same set difference operation as in set theory.

Example: To find students enrolled only in FRENCH but not in GERMAN, we write:

π(Student_Name)(FRENCH) - π(Student_Name)(GERMAN)

Student_Name
Ram
Mohan

Explanation: The only constraint in the Set Difference between two relations is that both relations must have the same set of Attributes.

5) Rename(ρ)

Rename operator basically allows you to give a temporary name to a specific relational table or to its columns. It is very useful when we want to avoid ambiguity, especially in complex Queries. Rename is a unary operation used for renaming attributes of a relation.

Example: We can rename an attribute B in relation R to D

A	B	C
1	2	4
2	2	3
3	2	3
4	3	4

ρ(D/B)R will rename the attribute 'B' of the relation by "D".

Output Table:

A	D	C
1	2	4
2	2	3
3	2	3
4	3	4

6) Cartesian Product(X)

Combines every row of one table with every row of another table, producing all the possible combination. It's mostly used as a precursor to more complex operation like joins. Let's say A and B, so the cross product between A X B will result in all the attributes of A followed by each attribute of B. Each record of A will pair with every record of B.

Relation A:

Name	Age	Sex
Ram	14	M
Sona	15	F
Kim	20	M

Relation B:

ID	Course
1	DS
2	DBMS

Output: If relation A has 3 rows and relation B has 2 rows, the Cartesian product A × B will result in 6 rows.

Name	Age	Sex	ID	Course
Ram	14	M	1	DS
Ram	14	M	2	DBMS
Sona	15	F	1	DS
Sona	15	F	2	DBMS
Kim	20	M	1	DS
Kim	20	M	2	DBMS

Explanation: If A has 'n' tuples and B has 'm' tuples then A X B will have 'n*m' tuples.

Derived Operators in Relational Algebra

Derived operators are built using basic operators and include operations like join, intersection, and division. These operators help perform more complex queries by combining basic operations to meet specific data retrieval needs.

1) **Join Operators** - Combine data from two or more relations based on a related attribute, allowing for more complex queries and data retrieval. Different types of joins include:

   1.1 Inner Join - Combines rows from two relations based on a matching condition and only returns rows where there is a match in both relations. If a record in one relation doesn't have a corresponding match in the other, it is excluded from the result. This is the most common type of join.

   a) Conditional Join: A conditional join is an inner join where the matching condition can involve any comparison operator like equals (=), greater than (>), etc.

   Example: Joining Employees and Departments on DepartmentID where Salary > 50000 will return employees in departments with a salary greater than 50,000

   b) Equi Join: An equi join is a type of conditional join where the condition is specifically equality (=) between columns from both relations.

   Example: Joining Customers and Orders on CustomerID where both relations have this column, returning only matching records.

   c) Natural Join: A natural join automatically combines relations based on columns with the same name and type, removing duplicate columns in the result. It's a more efficient way of joining.

   Example: Joining Students and Enrollments where StudentID is common in both, and the result contains only unique columns.

   1.2 Outer Join - Returns all rows from one relation, and the matching rows from the other relation. If there is no match, the result will still include all rows from the outer relation with NULL values in the columns from the unmatched relation.

   a) Left Outer Join: A left outer join returns all rows from the left relation and the matching rows from the right relation.

   If there is no match, the result will include NULL values for the right relation's attributes.

   Example: Joining Employees with Departments using a left outer join ensures all employees are listed, even those who aren't assigned to any department, with NULL values for the department columns.

   b) Right Outer Join:A right outer join returns all rows from the right relation and the matching rows from the left relation.

   If no match exists, the left relation's columns will contain NULL values.

   Example: Joining Departments with Employees using a right outer join includes all departments, even those with no employees assigned, filling unmatched employee columns with NULL.

c) Full Outer Join: A full outer join returns all rows when there is a match in either the left or right relation.

If a row from one relation does not have a match in the other, NULL values are included for the missing side.

Example: Joining Customers and Orders using a full outer join will return all customers and orders, even if there's no corresponding order for a customer or no customer for an order.

2) Set Intersection(∩)

Set Intersection basically allows to fetches only those rows of data that are common between two sets of relational tables. Set Intersection in relational algebra is the same set intersection operation in set theory.

Example: Consider the following table of Students having different optional subjects in their course.

Relation FRENCH

Student_Name	Roll_Number
Ram	01
Mohan	02
Vivek	13
Geeta	17

Relation GERMAN

Student_Name	Roll_Number
Vivek	13
Geeta	17
Shyam	21
Rohan	25

From the above table of FRENCH and GERMAN, the Set Intersection is used as follows:

π(Student_Name)(FRENCH) ∩ π(Student_Name)(GERMAN)

Output:

Student_Name
Vivek
Geeta

Explanation: The only constraint in the Set Difference between two relations is that both relations must have the same set of Attributes.

3) Division (÷)

The Division Operator is used to find tuples in one relation that are related to all tuples in another relation. It's typically used for "for all" queries.

Student_Course (Dividend Table):

Student_ID	Course_ID
101	C1
101	C2
102	C1
103	C1
103	C2

Course (Divisor Table):

Course_ID
C1
C2

Example: Query is to find students who are enrolled in all courses listed in the Course table. In this case, students must be enrolled in both C1 and C2.

Student_Course(Student_ID, Course_ID)÷ Course(Course_ID)

Output:

Student_ID
101
103

Advantages of Relational Algebra

- Simplicity and Precision: On the same note, Relational Algebra offers a simple and efficient methodology for how questions can be framed especially for a layman who has a crystal-clear understanding of the entire method.
- Optimization Potential: It makes optimization possible due to the procedural aspect, and shows how one can find the best way to execute the statement.
- Foundational for SQL: Most of the SQL commands can be traced to the Relational Algebra operations as will be seen from the following sections.

Disadvantages of Relational Algebra

- Complex for Complex Queries: Relational algebra is more or less effective when the query is highly complex because with increasing complexity, Relational Algebra analysis in full becomes a cumbersome exercise.
- Less Abstract: It is also procedural and thus it forces users to detail out the procedure of data access, which may not be easily understood by end users who are not very conversant with databases.

Difference between Relational Algebra and Relational Calculus

Relational Algebra and Relational Calculus are two preliminarily important topics in the study of DBMS. Both are standardized query languages used for querying databases but the two have different approaches and operations. Relational Algebra is imperative as it deals mainly with how the data is to be obtained whereas Relational Calculus is imperative-free as it is concerned with which data is to be obtained but not how. Knowledge of the differences between these two is critical to database professional as it impacts the way they design databases and optimize their queries.

Relational Calculus

Relational calculus is a non-procedural query language used in the context of relational algebra. It focuses on what data to retrieve, rather than how to retrieve it, making it different from relational algebra, which is procedural. In relational calculus, queries are expressed using logical formulas that describe the desired result, without specifying the exact steps to get there.

There are two types of Relational Calculus

1) Tuple Relational Calculus(TRC)
2) Domain Relational Calculus(DRC)

1) Tuple Relational Calculus(TRC)

Tuple Relational Calculus (TRC) is a non-procedural query language used to retrieve data from relational databases by describing the properties of the required data (not how to fetch it). It is based on first-order predicate logic and uses tuple variables to represent rows of tables.

Syntax: The basic syntax of TRC is as follows:

$$\{ t \mid P(t) \}$$

t: Tuple variable (row placeholder)

P(t): Predicate condition to satisfy

{}: Denotes a set of result tuples

Logical Operators in TRC:

∧: AND

∨: OR

¬: NOT

Quantifiers:

∃ t ∈ r (Q(t)) → There exists a tuple t in relation r satisfying predicate Q(t)

∀ t ∈ r (Q(t)) → For all tuples t in relation r, predicate Q(t) holds

For example, let's say we have a table called "Employees" with the following attributes:

Employee ID
Name
Salary
Department ID

To retrieve the names of all employees who earn more than $50,000 per year, we can use the following TRC query:

$$\{ t.Name \mid Employees(t) \wedge t.Salary > 50000 \}$$

Explanation:

- Employees(t) means t is a tuple from the Employees table.
- ∧ (AND) is used to add a condition on salary.
- The result is a set of tuples where each employee earns more than $50,000.

TRC is non-procedural - it specifies what data to retrieve, not how to retrieve it.

While expressive, TRC is more abstract and mainly used in academic or theoretical contexts, not practical database systems.

Tuple Relational Query

In Tuple Calculus, a query is expressed as

$$\{t \mid P(t)\}$$

- t represents the resulting tuples.
- P(t) is a predicate (a condition that must be true for t to be included in the resul
  P(t) may have various conditions logically combined with OR (∨), AND (∧), NOT(¬).
  It also uses quantifiers:
- ∃ t ∈ r (Q(t)) = ”there exists” a tuple in t in relation r such that predicate Q(t) is true.
- ∀ t ∈ r (Q(t)) = Q(t) is true "for all" tuples in relation r.

2) Domain Relational Calculus (DRC)

Domain Relational Calculus (DRC) is a formal query language for relational databases. It describes queries by specifying a set of conditions or formulas that the data must satisfy. These conditions are written using domain variables and predicates, and it returns a relation that satisfies the specified conditions.

Domain Relational Calculus is similar to Tuple Relational Calculus, where it makes a list of the attributes that are to be chosen from the relations as per the conditions.

$$\{<a1,a2,a3,.....an> \mid P(a1,a2,a3,.....an)\}$$

where

a1,a2,...an are the attributes of the relation and
P is the condition.

Tuple Relational Calculus Examples

Table Customer

Customer name	Street	City
John	A7	Chicago
Emma	B6	Houston
Olivia	D9	Dallas

Table Branch

Customer name	Street	City
John	A7	Chicago
Emma	B6	Houston
Olivia	D9	Dallas
Sophia	A5	Chicago

Table Branch

Branch name	Branch City
ABC	Chicago
DEF	Dallas
GHI	Houston
Branch name	Branch City

Table Account

Account number	Branch name	Balance
1111	ABC	50000
1112	DEF	10000
1113	GHI	9000
1114	ABC	7000

Table Loan

Loan number	Branch name	Amount
L33	ABC	10000
L35	DEF	15000
L49	GHI	9000
L98	DEF	65000

Table Borrower

Customer name	Loan number
John	L33
Emma	L49
Sophia	L98

Table Depositor

Customer name	Loan number
John	L33
Emma	L49
Sophia	L98
Customer name	Loan number

Table Depositor

Customer name	Account number
John	1111
Emma	1113
Olivia	1114

Example 1: Find the loan number, branch, and amount of loans greater than or equal to 10000 amount.

{t| t ∈ loan ∧ t[amount]>=10000}

Resulting relation:

Loan number	Branch name	Amount
L33	ABC	10000
L35	DEF	15000
L98	DEF	65000

In the above query, t[amount] is known as a tuple variable.

Example 2: Find the loan number for each loan of an amount greater or equal to 10000.

{ t[loan-number] | t ∈ loan ∧ t[amount] ≥ 10000 }

Resulting relation:

Loan number
L33
L35
L98

It retrieves the loan numbers of all tuples in the loan relation where the amount is greater than or equal to 10000.

Example 3: Find the names of all customers who have a loan and an account at the bank.

{ t.customer_name |
∃ s ∈ borrower (t.customer_name = s.customer_name)
∧ ∃ u ∈ depositor (t.customer_name = u.customer_name) }

Resulting relation:

Customer name
John
Emma

The query retrieves the names of customers who appear in both borrower and depositor relations, meaning they have both a loan and an account.

Example 4: Find the names of all customers having a loan at the "ABC" branch.

{ t[customer-name] | ∃ s ∈ borrower (t[customer-name] = s[customer-name]) ∧ ∃ u ∈ depositor (t[customer-name] = u[customer-name]) }

Resulting relation:

Customer name
John

Query retrieves the names of customers who have a loan (via borrower relation) and are linked to the specified branch, returning customers like Saurabh.

Comparison: TRC vs Relational Algebra

Feature	TRC	Relational Algebra
Type	Non-procedural	Procedural
Focus	What to retrieve	How to retrieve
Expression Style	Logical expressions	Set-based operators
Execution	Abstract, not directly executable	Directly convertible to query
Use in DBMS	Theoretical foundation	Basis for query execution

Key Characteristics of DRC:

- **Non-procedural**: Specifies what data to retrieve without describing the steps for retrieval.
- **Based on Predicate Calculus**: Utilizes logical expressions (predicates) to describe the query.
- **Relational Database Queries**: Primarily used for querying relational databases.

Components of Domain Relational Calculus (DRC)

1) **Domain Variables -** Domain variables represent the attributes (fields) that will appear in the resulting relation of the query. These variables are the **placeholders** that will hold the actual values in the result.
2) **Predicate -** A predicate is a **logical condition** or **formula** that the data must satisfy. It is expressed using comparison operators, connectives, and quantifiers.
   - **Comparison operators**: =, >, <, >=, <=, !=.
   - **Connectives**: AND, OR, NOT.
   - **Quantifiers**: FOR ALL, EXISTS.
3) **Quantifiers -** Quantifiers are used to express the scope of a query:
   Existential quantifier (∃): Denotes that there exists at least one instance that satisfies a condition.
   Universal quantifier (∀): Denotes that all instances in the domain satisfy a condition.
4) **Domains and Relations -** A domain refers to a specific set of values (like integers, strings) that a variable can take. A relation is a table in the database, and the tuples in the relation represent data that matches the query conditions.

Writing Queries in Domain Relational Calculus (DRC)

Writing a query in DRC involves describing the desired result by specifying the domain variables and predicates that define the conditions the data must meet. Below are examples:

Table 1: Customer

Customer Name	Street	City
Debomit	Kadamtala	Alipurduar
Sayantan	Udaypur	Balurghat
Soumya	Nutanchati	Bankura
Ritu	Juhu	Mumbai

Table 2: Loan

Loan Number	Branch Name	Amount
L01	Main	200
L03	Main	150
L10	Sub	90
L08	Main	60

Table 3: Borrower

Customer Name	Loan Number
Ritu	L01
Debomit	L08
Soumya	L03

Example 1: Finding Loans with Amount >= 100

To find all loans with an amount greater than or equal to 100, we use the following query in DRC:

$\{\prec l, b, a\succ \mid \prec l, b, a\succ \in loan \wedge (a \geq 100)\}$

Resulting relation:

Loan Number	Branch Name
L01	Main
L03	Main

This query specifies that the result will contain the loan number (l), branch name (b), and amount (a) from the loan relation where the amount is greater than or equal to 100.

Example 2: Finding Loan Numbers >= 150

To find loan numbers where the amount is greater than or equal to 150, we write:

$\{\prec l\succ \mid \exists\ b, a\ (\prec l, b, a\succ \in loan \wedge (a \geq 150))\}$

Resulting Relation:

Loan Number
L01
L03

Example 3: Finding Customers with Loans from "Main" Branch

To find the names of customers who have a loan at the "Main" branch, along with their loan amounts: $\{\prec c, a\succ \mid \exists\, l\, (\prec c, l\succ \in \text{borrower} \wedge \exists\, b\, (\prec l, b, a\succ \in \text{loan} \wedge (b = \text{“Main”})))\}$

Resulting relation:

Customer Name	Amount
Ritu	200
Debomit	60
Soumya	150

Advantages of Domain Relational Calculus (DRC)

- **Declarative Nature**: DRC allows the user to describe *what* data they need without having to specify *how* to retrieve it. This makes it easier to write queries and understand them.
- **Mathematical Foundation**: Being based on predicate logic, DRC provides a solid theoretical foundation for relational databases.
- **Non-Procedural**: Unlike procedural query languages, DRC focuses on describing the result, not the procedure to get it.
- **Flexible and Powerful**: It supports complex queries involving multiple relations, predicates, and conditions.

Limitations of Domain Relational Calculus (DRC)

- **Complexity**: DRC can be difficult to use for complex queries due to its reliance on logical formulas. It requires a good understanding of predicate calculus.
- **No Specification of Query Execution**: Since it is non-procedural, DRC does not specify how queries should be executed, which can sometimes lead to inefficiencies in query execution.
- **Lack of Optimization**: Unlike SQL, DRC does not provide built-in optimization mechanisms, meaning that users must rely on the DBMS for query optimization.
- **Less User-Friendly**: As a formal query language, DRC is not as user-friendly as SQL, which is more widely used and has a simpler syntax.

DRC (Domain Relational Calculus) Vs TRC (Tuple Relational Calculus)

Aspect	Domain Relational Calculus	Tuple Relational Calculus
Query Focus	Focuses on domain variables (attributes)	Focuses on tuple variables (rows)
Syntax	Describes queries using domain variables and predicates	Describes queries using tuple variables and predicates
Usage	More abstract; deals with individual attributes	Deals with entire rows (tuples) in a relation
Query Writing Style	Less intuitive; requires logical expressions involving attributes	More intuitive; uses variables representing tuples
Complexity	Can be more complex due to the abstraction of variables	Simpler to understand and write compared to DRC

While both DRC and TRC are non-procedural query languages that allow users to query relational databases, DRC focuses on individual attributes (domain variables), while TRC focuses on tuples (rows). TRC is more widely used because it is more intuitive and easier to understand.

.

Relational Algebra Vs Relational Calculus

Basis of Comparison	Relational Algebra	Relational Calculus
Language Type	It is a Procedural language.	Relational Calculus is a Declarative (non-procedural) language.
Procedure	Relational Algebra means how to obtain the result.	Relational Calculus means what result we have to obtain.
Order	In Relational Algebra, the order is specified in which the operations have to be performed.	In Relational Calculus, the order is not specified.
Domain	Relational Algebra is independent of the domain.	Relation Calculus can be domain-dependent because of domain relational calculus.
Programming language	Relational Algebra is nearer to a programming language.	Relational Calculus is not nearer to programming language but to natural language.
Inclusion in SQL	The SQL includes only some features from the relational algebra.	SQL is based to a greater extent on the tuple relational calculus.
Relationally completeness	Relational Algebra is one of the languages in which queries can be expressed but the queries should also be expressed in relational calculus to be relationally complete.	For a database language to be relationally complete, the query written in it must be expressible in relational calculus.
Query Evaluation	The evaluation of the query relies on the order specification in which the operations must be performed.	The order of operations does not matter in relational calculus for the evaluation of queries.
Database access	For accessing the database, relational algebra provides a solution in terms of what is required and how to get that information by following a step-by-step description.	For accessing the database, relational calculus provides a solution in terms as simple as what is required and lets the system find the solution for that.
Expressiveness	The expressiveness of any given language is judged using relational algebra operations as a standard.	The completeness of a language is measured in the manner that it is least as powerful as calculus. That implies relation defined using some expression of the calculus is also definable by some other expression, the language is in question.

===== &&&& =====